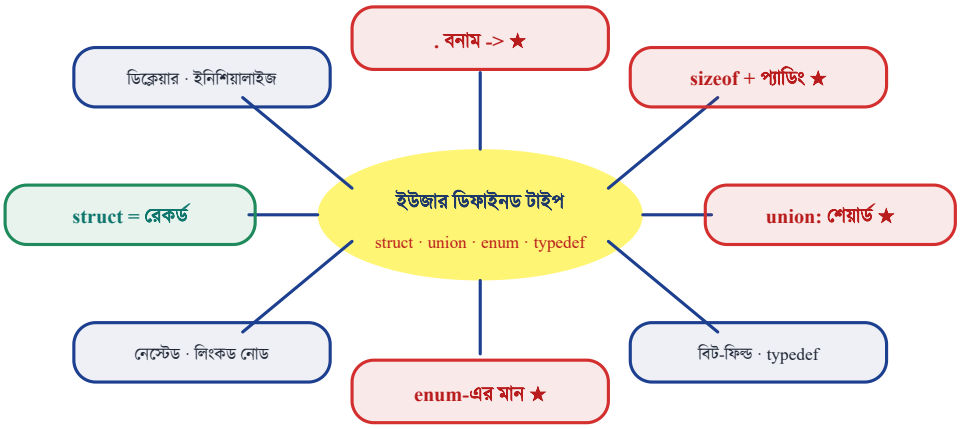
C-তে ইউজার ডিফাইনড ডেটা টাইপ

struct · . বনাম -> · প্যাডিং · বিট-ফিল্ড · union · enum · typedef: ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো লেসন এক পাতায়: লাল = বেশি আসে, সবুজ = ভিত্তি

সিলেবাসের যে অংশ

C সিলেবাসের লাইন: "**User defined data types: structures, unions, enumerations**"। পয়েন্টারের আলাদা লেসন আছে; ফাইলে স্ট্রাকচার লেখা-পড়া ফাইল I/O লেসনে।

হাই-ইল্ড তালিকা

① struct: প্রতিটি মেম্বারের নিজস্ব মেমোরি
② union: সব মেম্বার অফসেট 0-তে
③ sizeof(union) = সবচেয়ে বড়টি, রাউন্ড আপ
④ sizeof(struct) ≥ যোগফল (প্যাডিং)
⑤ p->x ≡ (*p).x; *p.x ≡ *(p.x)
⑥ s1 = s2 চলে; s1 == s2 এরর
⑦ enum শুরু 0; পরেরটি = আগের + 1
⑧ enum কনস্ট্যান্ট `int`
⑨ বিট-ফিল্ডের অ্যাড্রেস নেই
⑩ `typedef char *P; P a, b;` → দুটিই পয়েন্টার

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

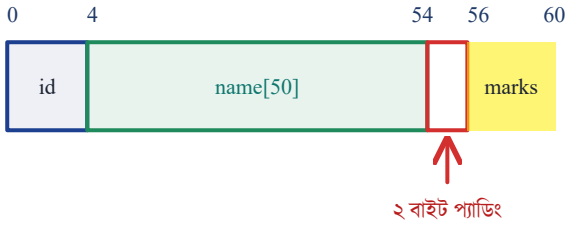
সূচিপত্র

✓ স্ট্রাকচার: ডিক্লেয়ার ও ইনিশিয়ালাইজ … পৃ. ২
✓ অ্যাক্সেস: . বনাম ->, ফাংশন … পৃ. ৩
✓ অ্যারে, নেস্টেড ও সেলফ-রেফারেন্সিয়াল … পৃ. ৪
✓ sizeof, প্যাডিং ও অ্যালাইনমেন্ট … পৃ. ৫
✓ বিট-ফিল্ড … পৃ. ৬
✓ ইউনিয়ন … পৃ. ৭
✓ এনুমারেশন … পৃ. ৮
✓ typedef + struct বনাম union বনাম enum … পৃ. ৯
✓ পরীক্ষার আগের রাতের পাতা … পৃ. ১০
✓ অনুশীলনী MCQ + লিখিত … পৃ. ১১

১ স্ট্রাকচার: ডিক্লারেশন, ডেফিনিশন, ইনিশিয়ালাইজেশন

সংজ্ঞা: **স্ট্রাকচার** (Structure, `struct`) হলো একটি ইউজার ডিফাইনড টাইপ, যা সাধারণত **ভিন্ন ভিন্ন** টাইপের সম্পর্কিত ভেরিয়েবলকে এক নামে একত্র করে। ভেতরের প্রতিটি ভেরিয়েবল একটি **মেম্বার** (Member, ফিল্ড)। মেম্বারগুলো **ডিক্লেয়ার করার ক্রমে** সাজানো থাকে এবং **প্রতিটি মেম্বারের নিজস্ব মেমোরি** থাকে। স্ট্রাকচার **টাইপ** (টেমপ্লেট) ডিক্লেয়ার করলে কোনো মেমোরি লাগে না; ওই টাইপের **ভেরিয়েবল** ডিফাইন করলে মেমোরি বরাদ্দ হয়।

```c
#include <stdio.h>
#include <string.h>
struct Student {          /* tag */
    int   id;
    char  name[50];
    float marks;
};                        /* ; needed */
int main(void) {
    struct Student s1;
    s1.id = 101;
    strcpy(s1.name, "Ruhul");
    s1.marks = 85.5f;
    printf("%d %s %.2f\n",
        s1.id, s1.name, s1.marks);
    return 0;  /* 101 Ruhul 85.50 */
}
```



4 + 50 + (2) + 4 = 60 বাইট

float শুরু হবে 4-এর গুণিতকে

মেমোরিতে একটি *struct Student*

ভেরিয়েবল তৈরির উপায়	কোড	মন্তব্য
টাইপের পরে	`struct Student s1, s2;`	C-তে `struct` কিওয়ার্ড লাগবেই
ডিক্লারেশনের সাথেই	`struct Student { … } s1, s2;`	টাইপ ও ভেরিয়েবল একসাথে
নামহীন (anonymous) struct	`struct { int x, y; } pt;`	ট্যাগ নেই: পরে আর ভেরিয়েবল নয়
পজিশনাল ইনিশিয়ালাইজার	`= {101, "Ruhul", 85.5f}`	মেম্বারের ক্রমে
ডেজিগনেটেড ইনিশিয়ালাইজার (C99)	`= {.marks = 85.5f, .id = 101}`	যেকোনো ক্রমে; name[] হয় ""
আংশিক / শূন্য	`= {101}` · `= {0}`	যে মেম্বার দেওয়া নেই সেটি **শূন্য**

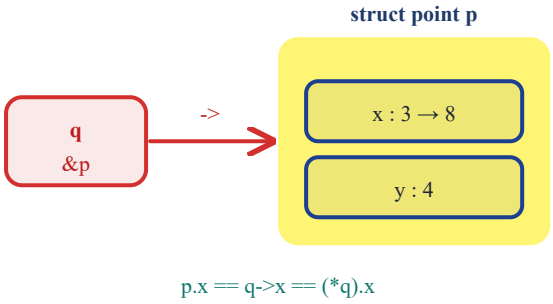
মনে রাখার কৌশল: "**struct হলো ফাঁকা ফর্ম; ভেরিয়েবল হলো পূরণ করা কপি।**" ফাঁকা ফর্মের নকশায় কাগজ লাগে না (মেমোরি নেই); প্রতিটি পূরণ করা কপিতে লাগে।

পরীক্ষার ফাঁদ: `s1.name = "Ruhul";` **কম্পাইল এরর**: অ্যারেতে অ্যাসাইন করা যায় না। `strcpy()` ব্যবহার করো, বা ডেফিনিশনের সময় ইনিশিয়ালাইজ করো। `}`-এর পরে `;` ভুলে গেলেও এরর।

সমাধানকৃত উদাহরণ ✍️ sizeof: 4-বাইট `int`/`float` ধরে: 4 + 50 = 54, *marks*-এর জন্য 56 পর্যন্ত প্যাড, + 4 → `sizeof(struct Student)` = **60**, 58 নয়।

২ মেম্বার অ্যাক্সেস: . বনাম ->; স্ট্রাকচার ও ফাংশন

সংজ্ঞা: **ডট অপারেটর** (Dot operator) `.` স্ট্রাকচার **ভেরিয়েবল** থেকে মেম্বার নেয়। **অ্যারো অপারেটর** (Arrow operator) `->` স্ট্রাকচারের **পয়েন্টার**-এর মাধ্যমে মেম্বার নেয়: `p->x` হুবহু `(*p).x`। ব্র্যাকেট জরুরি: ইউনারি `*`-এর চেয়ে `.`-এর প্রায়োরিটি বেশি, তাই `*p.x` মানে **`*(p.x)`**।



p.x == q->x == (*q).x

বাক্সে ডট, পয়েন্টার দিয়ে অ্যারো

```
struct point { int x, y; };
struct point p = {3, 4};
struct point *q = &p;
q->x = q->y * 2;
printf("%d %d", p.x, p.y);
/* Output: 8 4 */

void byVal(struct point a) { a.x = 0; }
void byPtr(struct point *a) { a->x = 0; }
/* byVal(p): p unchanged (copy)
   byPtr(&p): p.x becomes 0     */
```

এক্সপ্রেশন	অর্থ	এক্সপ্রেশন	অর্থ
`p->x`	`(*p).x`	`++p->x`	`++(p->x)`: মেম্বার বাড়ে
`*p.x`	`*(p.x)`: x-কে পয়েন্টার হতে হবে	`p++->x`	x পড়ে, তারপর p এগোয়
`&s.x`	`&(s.x)`: মেম্বারের অ্যাড্রেস	`*p->name`	নামের প্রথম অক্ষর

পুরো স্ট্রাকচারে অপারেশন	চলে?	কেন / কীভাবে
অ্যাসাইনমেন্ট `s2 = s1;`	**হ্যাঁ**	মেম্বার-বাই-মেম্বার কপি, ভেতরের অ্যারেসহ (একই টাইপ হলে)
তুলনা `s1 == s2`	**না**: কম্পাইল এরর	মেম্বার ধরে ধরে তুলনা; `memcmp` প্যাডিংও তুলনা করতে পারে
ফাংশনে পাঠানো / ফেরত দেওয়া	হ্যাঁ	বাই ভ্যালু পুরো struct কপি করে; বড় struct-এ পয়েন্টার পাঠাও

মনে রাখার কৌশল: **"বাক্স হাতে থাকলে ডট, বাক্সের দিকে আঙুল তুললে অ্যারো।"** ভেরিয়েবলের নাম → `.`; পয়েন্টার → `->`।

পরীক্ষার ফাঁদ: পাস-বাই-ভ্যালু কাজ করে **কপির** ওপর: `byVal(p)`-এর পর p অপরিবর্তিত। আর `::` কেবল C++-এর; C-তে মেম্বারের জন্য স্কোপ অপারেটর নেই।

সমাধানকৃত উদাহরণ ✍️ struct ফেরত:

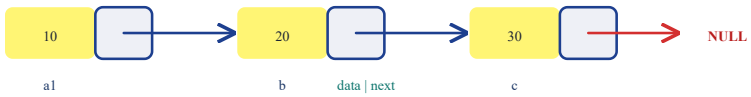
`struct point mid(struct point a, struct point b) { struct point m = {(a.x+b.x)/2, (a.y+b.y)/2}; return m; }` mid({2,4},{6,8}) → **{4, 6}**।

৩ স্ট্রাকচারের অ্যারে; নেস্টেড ও সেলফ-রেফারেন্সিয়াল স্ট্রাকচার

সংজ্ঞা: **স্ট্রাকচারের অ্যারে** (Array of structures) একই টাইপের অনেক রেকর্ড পাশাপাশি রাখে: `struct Student st[3];` → `st[0].id`, `st[2].name`। অ্যারের পয়েন্টার প্রতি `p++`-এ একটি **পুরো স্ট্রাকচার** এগোয় (struct-এর sizeof, ১ বাইট নয়)। **নেস্টেড স্ট্রাকচার**-এর একটি মেম্বার নিজেই struct; **সেলফ-রেফারেন্সিয়াল স্ট্রাকচার**-এর একটি মেম্বার **নিজের টাইপের পয়েন্টার**।

```
struct pt { int x, y; };   /* 8 B */
struct pt a[3] = {{1,2},{3,4},{5,6}};
struct pt *p = a;      /* &a[0] */
p += 2;                /* a[2]   */
printf("%d %d", p->x,
       (p-1)->y + a[0].x);
/* 5 5;  a at 1000 -> p = 1016 */
struct Date { int d, m, y; };
struct Emp {
    char name[20];
    struct Date dob;   /* nested */
} e, *pe = &e;
e.dob.y = 1995;  pe->dob.m = 6;
```

```
struct node {
    int data;
    struct node *next;  /* OK */
 /* struct node n;  -> error */
};
struct node c = {30, NULL};
struct node b = {20, &c};
struct node a1 = {10, &b};
printf("%d", a1.next->next->data);
/* Output: 30 */
```



তিনটি সেলফ-রেফারেন্সিয়াল নোড = একটি লিংকড লিস্ট

ধরন	মেম্বার	সাইজ (4-বাইট int, 8-বাইট পয়েন্টার)	অ্যাক্সেস
স্ট্রাকচারের অ্যারে	`struct pt a[3]`	3 × 8 = 24	`a[i].x`, `(p+i)->x`
নেস্টেড	Emp-এ `struct Date dob`	20 + 12 = 32	`e.dob.y`, `pe->dob.y`
সেলফ-রেফারেন্সিয়াল	`struct node *next`	4 + 4 প্যাড + 8 = **16** (32-বিটে 8)	`n->next->data`

মনে রাখার কৌশল: **"বাক্স নিজের মতো বাক্সের ঠিকানা রাখতে পারে, নিজেকে নয়।"** নিজেকে ধারণ করা struct অসীম বড় হতো।

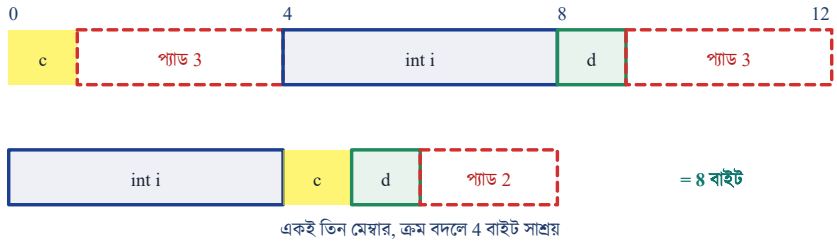
পরীক্ষার ফাঁদ: `struct pt *`-এ `p++` যোগ করে **8** বাইট (sizeof), ১ নয়। `e.dob.y`-তে দুটি ডটই লাগবে; `pe->dob->y` ভুল, কারণ *dob* পয়েন্টার নয়।

সমাধানকৃত উদাহরণ ✍️ হাঁটা: `for (p = a; p < a + 3; p++) sum += p->y;` → 2 + 4 + 6 = **12**। (আসল লিস্টে নোড আসে `malloc` থেকে: পয়েন্টার লেসন।)

৪ sizeof, প্যাডিং ও অ্যালাইনমেন্ট

সংজ্ঞা: CPU **অ্যালাইনড** (Aligned) অ্যাড্রেস থেকে সবচেয়ে দ্রুত পড়ে, তাই কম্পাইলার প্রতিটি মেম্বারকে তার **অ্যালাইনমেন্টের গুণিতক** অফসেটে বসায় এবং প্রয়োজনে অব্যবহৃত **প্যাডিং বাইট** (Padding) ঢোকায়। তারপর পুরো struct-কে তার **সবচেয়ে বড় মেম্বার-অ্যালাইনমেন্টের** গুণিতকে রাউন্ড আপ করে, যাতে এর অ্যারেও অ্যালাইনড থাকে। তাই **sizeof(struct) ≥ মেম্বারের যোগফল**।

ধরে নেওয়া (সাধারণ 64-বিট GCC/Clang/MSVC): char 1 · short 2 · int, float 4 · double, long long, পয়েন্টার 8 — প্রত্যেকে নিজের সাইজে অ্যালাইনড।



ওপরে: {char c; int i; char d;} = 12 · নিচে: {int i; char c; char d;} = 8

struct-এর মেম্বার (ক্রমানুসারে)	বিন্যাস (অফসেট)	sizeof
`char c; int i; char d;`	c@0, প্যাড 3, i@4, d@8, প্যাড 3	**12**
`int i; char c; char d;`	i@0, c@4, d@5, প্যাড 2	**8**
`char c; double d;`	c@0, প্যাড 7, d@8	16
`char c; short s; int i;`	c@0, প্যাড 1, s@2, i@4	8
`char a; double b; int c;`	a@0, প্যাড 7, b@8, c@16, শেষে প্যাড 4	24
`double b; int c; char a;`	b@0, c@8, a@12, শেষে প্যাড 3	16
`int a; char b[5];`	a@0, b@4..8, শেষে প্যাড 3	12
`char c; int *p;`	c@0, প্যাড 7, p@8 (32-বিটে: প্যাড 3, p@4)	16 (8)

মনে রাখার কৌশল: **"বড় পাথর আগে।"** মেম্বারগুলো বড় থেকে ছোট অ্যালাইনমেন্টে (double → int → short → char) সাজালে প্যাডিং প্রায় থাকে না।

পরীক্ষার ফাঁদ: প্যাডিং **ইমপ্লিমেন্টেশন-ডিফাইনড**। 32-বিট Linux-এ (i386) struct-এর ভেতরে double 4-অ্যালাইনড, তাই সেখানে `{char; double}` = 12। অপশনে প্যাডসহ মান না থাকলে প্রশ্নকর্তা **প্যাডিং ছাড়া** (সরল যোগফল) ধরেছেন।

সমাধানকৃত উদাহরণ ✍️ ধাপে ধাপে:

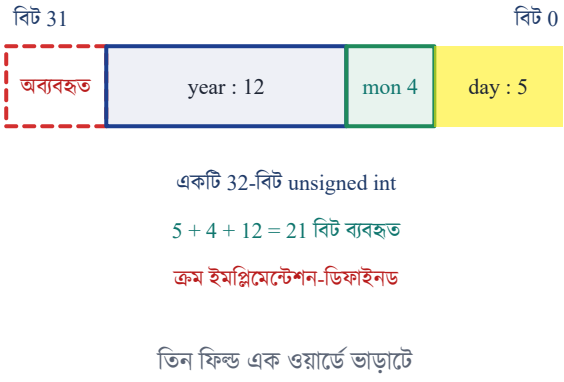
`{char a; double b; int c;}`: 1 → 8 পর্যন্ত প্যাড → +8 = 16 → +4 = 20 → 8-এর গুণিতকে = **24**।

প্যাকিং: `#pragma pack(1)` (বা GCC `__attribute__((packed))`) প্যাডিং সরায়: `{char c; int i; char d;}` হয় **6** বাইট, তবে আনঅ্যালাইনড অ্যাক্সেস ধীর। ফাইল ফরম্যাট ও নেটওয়ার্ক হেডারে ব্যবহৃত।

৫ বিট-ফিল্ড

সংজ্ঞা: **বিট-ফিল্ড** (Bit-field) হলো এমন struct মেম্বার যার প্রস্থ বিটে নির্দিষ্ট করা: `type name : width;`। টাইপ হতে হবে `int`, `signed int`, `unsigned int` বা `_Bool` (কম্পাইলার অন্যগুলোও দিতে পারে)। পাশাপাশি বিট-ফিল্ডগুলো একটি স্টোরেজ ইউনিটে (সাধারণত একটি `int`) ঠাসা হয়, ফলে ফ্ল্যাগ, ছোট কাউন্টার ও হার্ডওয়্যার রেজিস্টারে মেমোরি বাঁচে।

```
struct Date {
    unsigned day   : 5;  /* 0-31   */
    unsigned month : 4;  /* 0-15   */
    unsigned year  : 12; /* 0-4095 */
};          /* 21 bits -> 4 bytes */
struct Date d = {14, 10, 2026};
printf("%u/%u/%u", d.day,
       d.month, d.year);
/* 14/10/2026; 3 ints: 12 B */
struct { unsigned x : 3; } s;
s.x = 9;   /* 9 = 1001b, keeps 001 */
printf("%u", s.x);   /* 1 */
```



নিয়ম	উদাহরণ
অ্যাড্রেস নেই: & প্রয়োগ করা যায় না, তাই পয়েন্টার নেই, `scanf`-এ সরাসরি পড়াও নেই	`&d.day` → **কম্পাইল এরর**
বিট-ফিল্ডের অ্যারে হয় না; বিট-ফিল্ড `static` হতে পারে না	`unsigned f[4] : 2;` → এরর
প্রস্থ ≤ টাইপের বিট-সংখ্যা; প্রস্থ 0 কেবল নামহীন ফিল্ডে	`int a : 33;` (32-বিট int) → এরর
নামহীন `: 0` পরের ফিল্ডকে নতুন ইউনিটে ঠেলে দেয়	`{unsigned a:4; unsigned :0; unsigned b:4;}` = 8
ইউনিটের বাকি অংশে না ধরলে ফিল্ড নতুন ইউনিটে যায় (GCC)	`{unsigned a:30; unsigned b:4;}` = 8
একসাথে ধরলে সব ফিল্ড এক ইউনিটে	`{unsigned a:3, b:5, c:8;}` = 4
সাধারণ `int` ফিল্ড signed না unsigned, তা ইমপ্লিমেন্টেশন-ডিফাইনড	`int f:1` কেবল 0 ও −1 রাখতে পারে

মনে রাখার কৌশল: **"বিটরা এক ওয়ার্ডে ঘর ভাড়া নেয়।"** int ভরা না হওয়া পর্যন্ত ফিল্ডরা একই ঘরে ওঠে; `:0` ভাড়াটে বলে "পরের ফিল্ড, নতুন বাড়ি"।

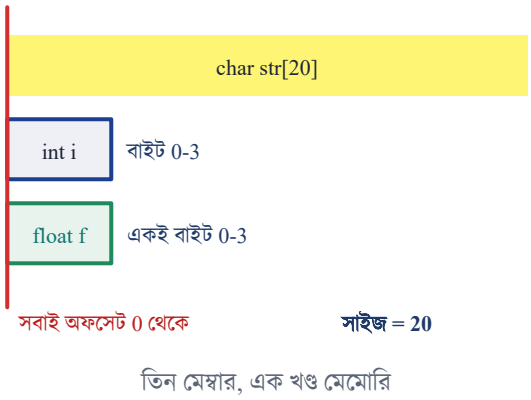
পরীক্ষার ফাঁদ: **unsigned** ফিল্ডে বড় মান রাখলে কেবল নিচের বিটগুলো থাকে (মান mod 2^{width}): 3 বিটে 9 → 1। **signed** ফিল্ডে ফল ইমপ্লিমেন্টেশন-ডিফাইনড (GCC: `int a:2; a = 3;` → −1)।

সমাধানকৃত উদাহরণ ✍️ রেঞ্জ: `unsigned m : 4` রাখে 0 থেকে $2^4 - 1 = \mathbf{15}$; ১২ মাসের জন্য যথেষ্ট। দিনের ফিল্ডে লাগে 5 বিট ($31 \le 31$)।

৬ ইউনিয়ন: এক মেমোরি, অনেক দৃষ্টিভঙ্গি

সংজ্ঞা: **ইউনিয়ন** (Union) স্ট্রাকচারের মতোই ডিক্লেয়ার করা হয়, কিন্তু **সব মেম্বার একই অ্যাড্রেস থেকে শুরু হয়** (অফসেট 0) এবং একে অপরের ওপর পড়ে। এর সাইজ **সবচেয়ে বড় মেম্বার, রাউন্ড আপ করা** — কঠোরতম মেম্বার-অ্যালাইনমেন্টের গুণিতকে। একটি মেম্বারে লিখলে অন্যগুলো মুছে যায়, তাই কেবল **সর্বশেষ লেখা** মেম্বারের মানই অর্থবহ। উদ্দেশ্য: কয়েকটি টাইপের মধ্যে একসাথে একটিই লাগলে মেমোরি বাঁচানো।

```
union Data {
    int   i;
    float f;
    char  str[20];
};                  /* sizeof = 20 */
union Data d;
d.i = 10;    printf("%d\n", d.i);
d.f = 3.14f; printf("%.2f\n", d.f);
strcpy(d.str, "Hello");
printf("%s\n", d.str);
/* 10 / 3.14 / Hello: each read
   comes right after its write */
d.f = 3.14f;
printf("%d", d.i); /* 1078523331 */
```



তিন মেম্বার, এক খণ্ড মেমোরি

ডিক্লারেশন (4-বাইট int/float, 8-বাইট double)	বড় মেম্বার	অ্যালাইনমেন্ট	sizeof
`union { int i; float f; char str[20]; }`	20	4	**20**
`union { char c[5]; int i; }`	5	4 → 5 রাউন্ড আপ	**8**
`union { double d; char c[9]; }`	9	8	16
`struct { int i; float f; char str[20]; }`	— (যোগফল)	4	28

```
/* endianness check */
union { int i; char c[4]; } u;
u.i = 1;
puts(u.c[0] == 1 ? "little-endian"
                 : "big-endian");
/* x86 / ARM default: little */
```

```
/* tagged union */
struct Value {
    enum { INT, FLT } tag;
    union { int i; float f; } v;
};  /* read v.i only if tag == INT */
```

মনে রাখার কৌশল: "**struct = দালানের আলাদা ফ্ল্যাট; union = এক ঘর, অনেক ভাড়াটে।**" কেবল সর্বশেষ ওঠা ভাড়াটেই আসলে সেখানে থাকে।

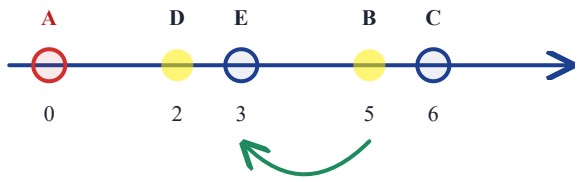
পরীক্ষার ফাঁদ: `union Data d = {10};` কেবল **প্রথম মেম্বার** (i) ইনিশিয়ালাইজ করে। অন্যটির জন্য `{.f = 2.5f}` (C99)। ইউনিয়নের সাইজ যোগফল **নয়**। 1078523331 = 0x4048F5C3, অর্থাৎ 3.14f-এর বিট প্যাটার্ন।

সমাধানকৃত উদাহরণ ✍️ বাইট: লিটল-এন্ডিয়ান PC-তে `u.i = 0x41424344;` হলে `u.c[0]` সবচেয়ে নিচের বাইট 0x44 → `printf("%c", u.c[0])` ছাপে **D**।

৭ এনুমারেশন (enum)

সংজ্ঞা: **এনুমারেশন** (Enumeration) হলো **নামযুক্ত ইন্টিজার কনস্ট্যান্ট** (এনুমারেটর) দিয়ে গঠিত ইউজার ডিফাইনড টাইপ। মান না দিলে প্রথম এনুমারেটর **0** এবং পরের প্রতিটি **আগেরটি + 1**। এনুমারেটর `int` টাইপের কনস্ট্যান্ট; এতে কোড পড়া সহজ হয় (`if (day == 5)`-এর বদলে `if (day == Friday)`)।

```
enum Week { Sunday, Monday, Tuesday,
   Wednesday, Thursday, Friday,
   Saturday };
enum Week today = Wednesday;
printf("%d", today);       /* 3 */
enum Month { Jan = 1, Feb, Mar, Apr };
                    /* 1 2 3 4 */
enum color { RED, GREEN = 5, BLUE };
printf("%d %d", RED, BLUE); /* 0 6 */
enum { A, B = 5, C, D = 2, E };
       /* 0, 5, 6, 2, 3 */
```



দেওয়া মান থেকে গোনা নতুন করে শুরু

enum { A, B = 5, C, D = 2, E }

নিয়ম	উদাহরণ / ফল
একই মান একাধিকবার চলে	`enum { LOW = 1, MIN = 1 };` ✓
ঋণাত্মক মান চলে	`enum { N = -2, M };` → M = −1
মান হতে হবে ইন্টিজার কনস্ট্যান্ট এক্সপ্রেশন	`enum { X = 2.5 };` → এরর; `C = B + 3` ✓
এনুমারেটর সাধারণ আইডেন্টিফায়ারের নেমস্পেস ভাগ করে	একই স্কোপে `enum A {X}; enum B {X};` → **এরর**
এনুমারেটর কনস্ট্যান্ট, ভেরিয়েবল নয়	`RED++;` বা `RED = 3;` → এরর
enum ভেরিয়েবল ভেতরে ইন্টিজারই (C)	`enum Week d = 10; d++;` C-তে কম্পাইল হয় (C++-এ নয়)
enum ভেরিয়েবলের `sizeof`	ইমপ্লিমেন্টেশন-ডিফাইনড, সাধারণত **4** (int-এর সমান)

বিষয়	`enum`	`#define`
কে সামলায়	কম্পাইলার (আসল টাইপ, ডিবাগারে নাম দেখায়)	প্রিপ্রসেসর: অন্ধ টেক্সট প্রতিস্থাপন
মান	কেবল ইন্টিজার; স্বয়ংক্রিয় নম্বর	যেকোনো কিছু (float, string); প্রতিটি হাতে লেখা
স্কোপ	ব্লক স্কোপ মানে	`#define` থেকে ফাইলের শেষ (বা `#undef`) পর্যন্ত

মনে রাখার কৌশল: **"শূন্যে শুরু, পরে এক এক করে বাড়ে, দেওয়া মান গোনা নতুন করে শুরু করায়।"**

পরীক্ষার ফাঁদ: নির্দিষ্ট মানের পর গোনা চলে **সেই মান** থেকে, অবস্থান থেকে নয়: `{RED, GREEN = 5, BLUE}`-এ BLUE হলো 6, 2 নয়।

সমাধানকৃত উদাহরণ ✍️: `enum { P = -1, Q, R = Q + 3, S };` → Q = 0, R = 3, S = **4**।

৮ typedef; struct বনাম union বনাম enum

সংজ্ঞা: **typedef** বিদ্যমান টাইপকে একটি নতুন **নাম** (অ্যালিয়াস) দেয়; নতুন টাইপ বা মেমোরি তৈরি করে না। স্ট্রাকচারের সাথে ব্যবহার করলে প্রতিবার `struct` লিখতে হয় না। struct **ট্যাগ** (`struct Student`) আর typedef নাম (`Student`) আলাদা নেমস্পেসে থাকে, তাই একই শব্দ দুটোতেই চলে।

```
typedef struct Student {
    int id;  char name[50];
} Student;          /* tag = alias */
Student s1;         /* no 'struct' */

typedef unsigned char BYTE;
typedef int (*Cmp)(const void *,
                   const void *);
Cmp f = compareInts; /* for qsort */
```

```
typedef char *PTR;
PTR a, b;      /* a, b: char *  */

#define PTR2 char *
PTR2 c, d;     /* char *c, d;
                  d is a char!  */
/* 64-bit: sizeof(b) = 8,
           sizeof(d) = 1        */
```

বিষয়	typedef	#define
কে প্রসেস করে	কম্পাইলার; স্কোপের নিয়ম মানে	প্রিপ্রসেসর; সরল টেক্সট প্রতিস্থাপন
পয়েন্টার অ্যালিয়াস, দুটি ভেরিয়েবল	**দুটিই পয়েন্টার**	কেবল প্রথমটি পয়েন্টার

বৈশিষ্ট্য	struct	union	enum
কী রাখে	একসাথে অনেক মেম্বার	একবারে একটি মেম্বার	নামযুক্ত int কনস্ট্যান্ট
মেমোরি	প্রতিটি মেম্বারের আলাদা	শেয়ার্ড, সবাই অফসেট 0-তে	ভেরিয়েবল = একটি int
sizeof	যোগফল + প্যাডিং	সবচেয়ে বড়টি, রাউন্ড আপ	সাধারণত 4
ইনিশিয়ালাইজার	সব মেম্বার ক্রমানুসারে	কেবল প্রথম মেম্বার (বা ডেজিগনেটেড)	একটি এনুমারেটর / int
অ্যাক্সেস	`.` ও `->`	`.` ও `->`	নাম সরাসরি
সাধারণ ব্যবহার	রেকর্ড, লিস্ট নোড	ভ্যারিয়েন্ট, টাইপ পানিং, মেমোরি সাশ্রয়	স্টেট, মেনু, দিন

আউটপুট কী? (64-বিট, লিটল-এন্ডিয়ান)

① কপি, অ্যালিয়াস নয়: `typedef struct { int x; } T; T a = {5}, b; b = a; b.x++; printf("%d %d", a.x, b.x);` → **5 6** (একটি কপি)।

② struct-এর সাইজ: `struct S { int a; char b; };` `sizeof(struct S)` → **8** (4 + 1 + শেষে 3 প্যাড)।

মনে রাখার কৌশল: "**typedef নাম বদলায়, #define লেখা বদলায়।**" typedef কম্পাইলার বোঝে; প্রিপ্রসেসর কেবল টেক্সট বসিয়ে দেয়।

পরীক্ষার ফাঁদ: typedef ছাড়া C-তে লিখতে হয় `struct Student s;` — শুধু `Student s;` C-তে এরর (C++-এ চলে)। `fwrite` দিয়ে পুরো রেকর্ড লেখা: ফাইল I/O লেসন।

পরীক্ষার আগের রাতের পাতা

তথ্য পাতা:

struct: প্রতি মেম্বারের নিজস্ব মেমোরি

sizeof(struct) = মেম্বার + প্যাডিং

`p->x` ≡ `(*p).x`

`s2 = s1` ✓ · `s1 == s2` ✗

enum: 0, তারপর আগের + 1

বিট-ফিল্ড: & নেই, অ্যারে নেই

struct নিজের পয়েন্টার রাখতে পারে

union: সব মেম্বার অফসেট 0-তে

sizeof(union) = সবচেয়ে বড়টি, রাউন্ড আপ

`*p.x` ≡ `*(p.x)`

union ইনিশিয়ালাইজ → কেবল প্রথম মেম্বার

enum কনস্ট্যান্টের টাইপ = `int`

unsigned n-বিট ফিল্ড: 0 … $2^n - 1$

`p++` সরে sizeof(struct) বাইট

যে সাইজগুলো জানতেই হবে (4-বাইট int, 8-বাইট double/পয়েন্টার)

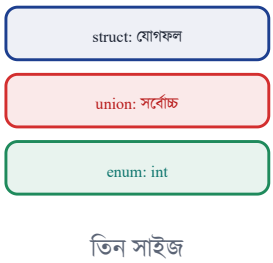
টাইপ	সাইজ	টাইপ	সাইজ
`{char; int; char}`	12	`{int; char; char}`	8
`{char; double}`	16	`{int id; char name[50]; float m;}`	60
`union {int; float; char[20]}`	20	`union {char[5]; int}`	8
`{int data; struct node *next;}`	16	`{unsigned a:3, b:5, c:8;}`	4

MCQ-র জন্য সেরা ১০টি এক-লাইনের কথা

1. struct মেম্বার ডিক্লারেশনের ক্রমে থাকে
2. struct টাইপ ডিক্লেয়ারে মেমোরি লাগে না
3. struct-এর ভেতরের অ্যারেতে `strcpy`, = নয়
4. ডেজিগনেটেড ইনিশিয়ালাইজার: `{.id = 1}` (C99)
5. পাস বাই ভ্যালু → ফাংশন কপি বদলায়
6. union একবারে একটি বৈধ মেম্বার রাখে
7. এন্ডিয়াননেস পরীক্ষা: `union {int; char[4]}`
8. enum-এ একই মান ও ঋণাত্মক মান বৈধ
9. দুই enum-এ একই এনুমারেটর → এরর
10. `typedef char *P; P a, b;` → দুটি পয়েন্টার

সেরা ফাঁদ: sizeof ≠ মেম্বারের যোগফল · GREEN = 5-এর পর BLUE হলো 6 · `*p.x`-এ ব্র্যাকেট লাগে · struct-এ == নেই · `&bitfield` অবৈধ · C-তে শুধু `Student s;` লিখতে typedef লাগে · `#define` পয়েন্টার অ্যালিয়াসে একটিই পয়েন্টার।

সব কৌশল একসঙ্গে: ফাঁকা ফর্ম বনাম পূরণ করা কপি · হাতে থাকলে ডট, আঙুল তুললে অ্যারো · বাক্স রাখে বাক্সের ঠিকানা, নিজেকে নয় · বড় পাথর আগে · বিটরা ঘর ভাড়া নেয় · এক ঘর, অনেক ভাড়াটে · শূন্যে শুরু, পরে এক এক করে · typedef নাম বদলায়, #define লেখা বদলায়।



তিন সাইজ

✓ অনুশীলনী MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. 4-বাইট int ও 8-বাইট double হলে `union { int i; double d; char c[10]; }`-এর সাইজ:
ক) 22 খ) 10
গ) 16 ঘ) 8

২. পয়েন্টারের মাধ্যমে স্ট্রাকচার মেম্বারে পৌঁছায় কোন অপারেটর?
ক) . খ) ->
গ) * ঘ) &

৩. `enum { X, Y = 4, Z };`-এ *Z*-এর মান:
ক) 2 খ) 4
গ) 6 ঘ) 5

৪. C-তে পুরো স্ট্রাকচারে কোন অপারেশন চলে না?
ক) `==` দিয়ে তুলনা খ) অ্যাসাইনমেন্ট
গ) ফাংশনে পাঠানো ঘ) ফাংশন থেকে ফেরত

৫. সাধারণ 32/64-বিট কম্পাইলারে `sizeof(struct { char c; int i; char d; })`:
ক) 6 খ) 8
গ) 12 ঘ) 16

৬. বিট-ফিল্ড মেম্বারের অ্যাড্রেস:
ক) নেওয়া যায় না খ) সবসময় জোড়
গ) struct-এর অ্যাড্রেস ঘ) কাস্ট লাগে

৭. `typedef char *P; P a, b;`-এর পর *b*-এর টাইপ:
ক) char খ) int
গ) char ** ঘ) char *

৮. C-তে একটি স্ট্রাকচার ধারণ করতে পারে:
ক) নিজ টাইপের মেম্বার খ) নিজ টাইপের পয়েন্টার
গ) ফাংশনের বডি ঘ) কেবল int মেম্বার

৯. ইউনিয়নের মেম্বাররা:
ক) একই অ্যাড্রেস শেয়ার করে খ) আলাদা মেমোরি পায়
গ) লিংকড লিস্ট বানায় ঘ) সবই পয়েন্টার হয়

১০. `*p.x` এক্সপ্রেশনটি পার্স হয়:
ক) `(*p).x` খ) `p->x`
গ) `&p.x` ঘ) `*(p.x)`

উত্তরমালা → ১-গ · ২-খ · ৩-ঘ · ৪-ক · ৫-গ · ৬-ক · ৭-ঘ · ৮-খ · ৯-ক · ১০-ঘ

লিখিত প্রশ্ন: উদাহরণসহ স্ট্রাকচার ও ইউনিয়নের পার্থক্য লেখো। `{int i; float f; char s[10];}` একবার struct ও একবার union হিসেবে ডিক্লেয়ার করলে দুটির সাইজ বের করো (4-বাইট int/float)। ট্রাফিক লাইটের অবস্থার জন্য একটি enum লেখো ও এর মানগুলো দেখাও। ১০ নম্বর

১ **পার্থক্য (৪)** — struct: প্রতিটি মেম্বারের নিজস্ব মেমোরি, সবাই একসাথে বৈধ, সাইজ = যোগফল + প্যাডিং; union: মেম্বাররা অফসেট 0-তে একে অপরের ওপর, কেবল সর্বশেষ লেখাটি বৈধ, সাইজ = সবচেয়ে বড়টি রাউন্ড আপ; দুটিতেই `.`/`->`। ৪

২ **সাইজ (৩)** — struct: 4 + 4 + 10 = 18 → 4-এর গুণিতকে = **20**; union: সবচেয়ে বড় 10 → 4-এর গুণিতকে = **12**। ৩

৩ **enum (৩)** — `enum Light { RED, YELLOW, GREEN };` → 0, 1, 2; `enum Light s = GREEN; printf("%d", s);` ছাপে 2; এনুমারেটর int কনস্ট্যান্ট। ৩

পরীক্ষার আগে ✓

- ✓ প্যাডিংসহ sizeof বের করতে পারি
- ✓ ঠিকভাবে `.` বা `->` বাছতে পারি
- ✓ enum-এর মান ট্রেস করতে পারি
- ✓ union-এর সাইজ ও ইনিশিয়ালাইজের নিয়ম জানি
- ✓ বিট-ফিল্ডের সীমাবদ্ধতা জানি
- ✓ typedef/#define ফাঁদ ধরতে পারি